

ETC — El Trío Calavera

ETC — El Trío Calavera

Configuración de agentes y skills para OpenCode, el entorno de codificación con IA. Este repo alberga a **ETC — El Trío Calavera**, tres agentes especializados que forman un equipo de desarrollo completo, más 17 skills complementarias.

«Uno escribe el código, otro lo cura, el tercero lo cuestiona. Juntos: ETC — El Trío Calavera.»

ETC — Las 3 Skills/Agents Principales

ETC	Personaje	Rol	Frase
	El Maestro	TDD Orchestrator	“Sin tests no hay commit.”
	Bug Doctor	Diagnóstico Forense	“Sin repro, no toco el código.”
	El de las Gafas	Domain Moderator	“Una palabra ambigua hoy es un bug mañana.”

El Maestro — TDD Orchestrator

Orquesta el ciclo completo de desarrollo guiado por pruebas: **INIT → PLAN → RED → GREEN → REFACTOR → REVIEW → COMMIT**. No escribe código directamente — guía el proceso con disciplina, exigiendo 90%+ de cobertura y 0 errores de análisis estático.

Origen: [morodomi/tdd-skills](https://github.com/morodomi/tdd-skills)

Bug Doctor — Diagnóstico Disciplinado de Bugs

Diagnostica bugs complejos con un método forense de 6 fases. Su principio fundamental: sin un loop de feedback determinista, mirar código es perder el tiempo. Reproduce, minimiza, formula hipótesis falsificables, instrumenta, corrige y blinda con tests de regresión.

Origen: [morodomi/tdd-skills](https://morodomi.com/tdd-skills)

El de las Gafas — Domain Moderator

Moderador de dominio que ve lo que otros pasan por alto: términos ambiguos, contradicciones entre código y discurso, decisiones no documentadas. Actualiza `CONTEXT.md` y crea ADRs en vivo mientras las decisiones cristalizan. Aplica principios de Domain-Driven Design.

Origen: [mattpocock/skills](https://mattpocock.com/skills)

Estructura de Archivos

Para implementar estas skills en tu proyecto, necesitas esta estructura:

```
tu-proyecto/
├── .opencode/
│   ├── .gitignore           # node_modules, package.json, etc.
│   ├── agents/              # Agentes (subagentes autónomos)
│   │   ├── el-maestro.md
│   │   ├── bug-doctor.md
│   │   └── el-de-las-gafas.md
│   └── skills/               # Skills (instrucciones
especializadas)
│       ├── el-maestro/
│       │   └── SKILL.md
│       ├── bug-doctor/
│       │   └── SKILL.md
│       ├── el-de-las-gafas/
│       │   ├── SKILL.md
│       │   ├── ADR-FORMAT.md
│       │   └── CONTEXT-FORMAT.md
│       └── ... (17 skills adicionales)
├── opencode.json             # Configuración de permisos
└── README.md
```

Disponibilidad Global

Para que las skills y agentes estén disponibles en **todos** tus proyectos, copia el contenido de `.opencode/` a `~/.config/opencode/`:

```
cp -r .opencode/skills/* ~/.config/opencode/skills/
cp -r .opencode/agents/* ~/.config/opencode/agents/
```

OpenCode carga skills y agentes desde: 1. **.opencode/ del proyecto** (específico del proyecto, tiene prioridad) 2. **~/.config/opencode/** (global, disponible en todos los proyectos)

Formato de los Archivos

Skill: `SKILL.md`

Cada skill vive en su propio directorio bajo `.opencode/skills/<nombre>/` con un archivo `SKILL.md`. El formato incluye frontmatter YAML:

```
---
name: nombre-de-la-skill
description: Qué hace y cuándo usarla
license: MIT
compatibility: opencode
---

# Título de la Skill

Instrucciones completas para el modelo...
```

Reglas: - El directorio se nombra igual que la skill - El archivo debe llamarse exactamente `SKILL.md` - El frontmatter es obligatorio: `name`, `description`, `license`, `compatibility` - Las skills se cargan al inicio de la conversación mediante la herramienta `skill`

Agent: `.md` con frontmatter

Cada agente vive en `.opencode/agents/<nombre>.md`. El formato incluye frontmatter YAML con `mode: subagent`:

```
---
description: Descripción de cuándo usar este agente
mode: subagent
---

# Nombre del Agente
```

Instrucciones completas de personalidad, misión, reglas y flujo de trabajo...

Reglas: - El archivo se nombra igual que el agente: `el-maestro.md` -
mode: subagent es obligatorio para que OpenCode lo reconozca como agente autónomo - Los agentes se invocan con `@nombre-del-agente` en el chat

Configuración: `opencode.json`

Controla los permisos de las skills:

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "skill": {
      "*": "allow"
    }
  }
}
```

Cómo se Usan

Hay tres formas de invocar skills y agentes en OpenCode:

Método	Sintaxis	Aplica a	Ejemplo
@mención	<code>@el-maestro</code>	Agents	<code>@el-maestro</code> implementa login con JWT
/comando	<code>/el-maestro</code>	Skills y Agents	<code>/bug-doctor</code>
Lenguaje natural	Describir la tarea	Skills	“Necesito diagnosticar un bug intermitente”

Diferencia clave: Skill vs Agent

	Skill	Agent
Propósito	Instrucciones para Plan/Build	Subagente autónomo con personalidad
Invocación		<code>@agente</code>

	Skill	Agent
	/skill o lenguaje natural	
Contexto	Se inyecta en la conversación actual	Ejecuta en un contexto aislado
Archivo	.opencode/skills/<nombre>/SKILL.md	.opencode/agents/<nombre>.md

Las 3 skills principales existen en **ambos formatos** — como skill (para usar con /el-maestro o lenguaje natural) y como agent (para usar con @el-maestro). Esto permite máxima flexibilidad.

Skills Complementarias

Además de las 3 principales, este repo incluye 17 skills adicionales para diferentes especialidades:

Skill	Especialidad
accessibility-auditor	Auditoría de accesibilidad WCAG
api-tester	Testing y validación de APIs
backend-architect	Diseño de sistemas escalables
code-reviewer	Revisión de código constructiva
codebase-onboarding-engineer	Onboarding a codebases desconocidos
database-optimizer	Optimización de esquemas y queries
devops-automator	CI/CD e infraestructura como código
frontend-developer	Desarrollo web moderno (React/Vue/Angular)
git-workflow-master	Estrategias de branching y conventional commits
language-translator	Traducción español ↔ inglés en tiempo real
performance-benchmark	Medición y optimización de rendimiento
product-manager	Ciclo completo de producto (descubrimiento a GTM)
rapid-prototyper	Prototipado rápido y MVPs
security-engineer	Modelado de amenazas y código seguro

Skill	Especialidad
software-architect	Diseño de sistemas y DDD
sre	SLOs, error budgets, observabilidad
technical-writer	Documentación para desarrolladores

Cada una sigue el mismo formato `SKILL.md` y puede copiarse a `~/.config/opencode/skills/` para disponibilidad global.

Cómo Replicar Esto en Tu Proyecto

Paso 1: Crear la estructura

```
mkdir -p .opencode/skills .opencode/agents
```

Paso 2: Copiar las skills deseadas

```
# Desde este repo
cp -r .opencode/skills/el-maestro tu-proyecto/.opencode/skills/
cp -r .opencode/skills/bug-doctor tu-proyecto/.opencode/skills/
cp -r .opencode/skills/el-de-las-gafas tu-proyecto/.opencode/skills/

# Copiar los agents
cp .opencode/agents/el-maestro.md tu-proyecto/.opencode/agents/
cp .opencode/agents/bug-doctor.md tu-proyecto/.opencode/agents/
cp .opencode/agents/el-de-las-gafas.md tu-proyecto/.opencode/agents/
```

Paso 3: Configurar `opencode.json`

```
{
  "$schema": "https://opencode.ai/config.json",
  "permission": {
    "skill": {
      "*": "allow"
    }
  }
}
```

Paso 4: (Opcional) Disponibilidad global

```
cp -r .opencode/skills/* ~/.config/opencode/skills/
cp -r .opencode/agents/* ~/.config/opencode/agents/
```

Paso 5: Usar en OpenCode

Abre OpenCode en tu proyecto y escribe `@el-maestro` para iniciar un ciclo TDD, `@bug-doctor` para diagnosticar un bug, o `@el-de-las-gafas` para estresar tu modelo de dominio.

Licencia

MIT — heredada de [morodomi/tdd-skills](https://github.com/morodomi/tdd-skills) y [mattpocock/skills](https://github.com/mattpocock/skills).